How the Reference-Data Sync Actually Works

Last Updated: 2026-09-08

This document explains, in plain language, what the reference-data sync does and how it decides what to change. It is written for someone who wants to review or approve the logic without reading the source code. The technical companion document is `Database/SyncTool/README.md`; the actual engine lives in `Database/SyncTool/mtm_sync/`.

What the Sync Is

The sync copies a small, carefully chosen set of "reference data" (lookup and seed tables) from the **test** database into a chosen target database. Today that set is the dunnage reference tables. It is a one-way copy with strict rules: it never mirrors test over core, never deletes anything, and never touches any table that is not on its allowlist.

The target is chosen when you run it. In the app, the **Database Config** page offers two targets:

- **Core** — the live `mtm_receiving_application` database.
- **Disposable copy** — a throwaway working copy named `mtm_receiving_application_backup_test` used to prove the sync is safe before it ever touches live data.

The source is always the test database (`mtm_receiving_application_test`).

The Four Databases

Role	Database	Who may write it
Core (live)	`mtm_receiving_application`	Only after an automatic backup plus a typed confirmation

Role	Database	Who may write it
Test (source)	`mtm_receiving_application_test`	Never changed by the sync (read only here)
Disposable copy	`mtm_receiving_application_backup_test`	Yes — a working copy made fresh from core
Live-backup snapshot	`mtm_receiving_application_backup`	Created automatically before each live run

The One-Page Rule Book

Every decision the sync is allowed to make is spelled out in one checked-in file: `Database/SyncTool/sync_policy.yml`. It contains:

- The list of tables the sync may write, with direction `test -> core`.
- The business key for each table (for example `type_name` for `dunnage_types`) — never the auto-increment `id`.
- The columns that are never overwritten from test (audit columns like `created_by` and `modified_by`).
- The write mode for each table (`upsert` or `insert-if-missing`).
- A long list of tables that are explicitly excluded, with a reason.

Only the five dunnage reference tables are currently on the allowlist:

Table	Key	Mode	Notes
`dunnage_types`	`type_name`	upsert	Master dunnage container types
`dunnage_parts`	`part_id`	upsert	Dunnage part master data
`dunnage_quantity_types`	`quantity_type`	upsert	Quantity label headers
`dunnage_requires_inventory`	`part_id`	upsert	Returnable dunnage tracked in ERP
`dunnage_non_po_entries`	`value`	insert-if-missing	Saved non-PO reasons

Everything else — history, label data, users, security roles, settings, reporting, Volvo and receiving records, and so on — is on the excluded list and is never written.

How a Row Is Decided (The Core Logic)

For each allowlisted table the sync reads the test rows, compares them to the target rows by the table's business key, and classifies every row. This happens twice per run: first to build a plan (read-only), then to carry it out.

1. Rows that exist **only in test** are added to the target (inserted).
2. Rows that exist **only in target** are left completely alone. The sync never deletes a row, even if test does not have it.
3. Rows that exist **in both** follow the table's mode:

Mode	Behavior for a row found in both databases
`upsert`	The non-ignored columns in the target are updated to match test
`insert-if-missing`	Nothing happens — the target row is left as it is

4. Rows whose business key is empty, or that point at a record that no longer exists inside test itself, are skipped (counted as "skipped", never guessed).
5. Ignored columns (audit columns such as `created_by`) are never compared and never updated on rows that already exist. They may only be filled from the source when a brand-new row is being created and the target requires a value.

A shorthand for the whole thing: after a successful run, the target contains the **union** of its own rows and test's rows, where shared rows follow the table's mode.

Why a Second Run Changes Nothing

Because rows are matched on the business key and the target is only ever brought up to test's values, running the sync again on an already-updated target finds no inserts and no updates. After every execution the tool re-plans and verifies that a second run would change zero rows. If it would not, the run is reported as a failure rather than silently accepted.

Linking Rows When the "id" Columns Differ

Some child rows point at a parent row using the parent's auto-increment `id` (for example `dunnage_parts.type_id` points at `dunnage_types.id`). The `id` numbers are not the same in test and core, and the parent row may have just been inserted in the same run. To keep the link correct, the sync:

- Reads the parent's natural key in test (for example the type name).
- Writes the child's link through a lookup on the parent's natural key in the target database at the moment of the write.

The policy file marks these columns under `fk_map`. The result is that foreign keys stay valid even when the databases use different `id` numbers and even when the parent row is brand new.

Order of Work

Tables are processed in an order that respects their relationships — parents before children — using the real foreign-key relationships on the target. If the tables ever formed a loop, the tool falls back to the order already listed in the policy file (which is written parents-first) so that no table is ever processed before the table it depends on.

Running Against the Live Database (The Guarded Path)

When the target is **core**, the tool takes these steps in order:

1. Makes a full automatic snapshot of the live database into `mtm_receiving_application_backup` (all tables, data, views, procedures, functions, and triggers). This is not optional.
2. Prints the plan and asks for a typed confirmation before writing anything. If you do not confirm, nothing is written and the snapshot is kept for reference.
3. Runs all inserts and updates inside a single transaction. Either every table is applied, or none is.
4. If anything fails, the transaction is rolled back and the live database is restored from the snapshot taken in step 1.

5. On success it re-checks that a second run would change zero rows.

Running Against the Disposable Copy

The disposable copy is the safe rehearsal space. If it does not exist when you start a run against it, the tool first creates it fresh from core, then performs the sync into it. Running there never touches live data, so no snapshot or typed confirmation is required.

How We Prove It Is Safe

The `verify-copy` workflow automates the full safety rehearsal and prints a PASS or FAIL for every check:

1. Create the disposable copy from core.
2. Show the plan, write a reviewable SQL file (not executed), and run the sync on the copy.
3. Verify three things:
   - The copy's schema is identical before and after the run (the sync only changes data, never structure).
   - Every allowlisted table in the copy equals the union of core rows and test rows (core-only rows survive, test-only rows are added, shared rows follow the mode).
   - Excluded tables in the copy are byte-for-byte unchanged from core.
4. Run the sync a second time and assert zero changes.
5. Drop the disposable copy.

Only a full PASS authorizes a run against live core.

What the Run Output Means

A typical output after a run looks like this:

```
Planning sync mtm_receiving_application_test -> mtm_receiving_application_backup_test ...

Table                              +insert   ~update   same    skip
-------------------------------------------------------------------
dunnage_non_po_entries             0         0         7       0
dunnage_types                      2         0         21      0
dunnage_parts                      5         9         119     0
...
Totals: 9 insert(s), 9 update(s). Second run will change 0 rows (idempotent).

Executing ...
  dunnage_types: 2 row(s) changed
Committed.

Validating after execution (idempotency) ...
Validation OK: a second run would change 0 rows.
```

The columns are read as follows:

- `+insert` — rows that existed in test but not in the target and will be added.
- `~update` — rows that exist in both and (for `upsert` tables) had values that differ and will be brought up to test's values.
- `same` — rows already identical; nothing will be done.
- `skip` — rows the tool deliberately left alone (empty key or a broken link inside the source data).

Where the Decisions Live (For Reviewers)

To review or change what the sync is allowed to do, look at these places:

- `Database/SyncTool/sync_policy.yml` — the single rule book: allowlist, keys, ignored columns, modes, and exclusions.
- The generated SQL file from a `generate-sql` run — the exact, ordered, reviewable statements, written without executing anything.
- `Database/SyncTool/mtm_sync/sync.py` — the planning and execution engine.
- `Database/SyncTool/mtm_sync/cli.py` — the commands and the guarded core path.
- `Database/SyncTool/mtm_sync/verify.py` — the PASS/FAIL safety checks.

See Also

- `Database/SyncTool/README.md` — technical usage and commands.
- `Database/SyncTool/sync_policy.yml` — the sync rule book.
- `Database/Scripts/BackupLiveDatabase/` — the standalone live-database clone tool (mysqldump based) and its documentation.
- `Prompt.md` at the repository root — the working spec this tool implements.